

Aula 02 - Meu Primeiro Repositório e Linha do Tempo

Descrição: Criação do primeiro repositório local utilizando comandos do Git. Os estudantes aprenderão a realizar o versionamento na prática, monitorando alterações em seus arquivos e criando um histórico seguro de "commits" para acompanhar a evolução do projeto.

Meu Primeiro Repositório e Linha do Tempo

Objetivo da Aula: Ensinar o estudante a criar o seu primeiro repositório Git local do absoluto zero, compreender a área de preparação (*staging area*), configurar sua identidade no Git e registrar sua primeira versão estável (o commit inicial) na linha do tempo do projeto.

1. Introdução Teórica: O que é um Repositório e uma Linha do Tempo?

Antes de abrir o terminal, é essencial que o estudante compreenda a metáfora da "linha do tempo":

- **O Repositório** é a pasta do seu projeto monitorada pelo Git. É onde o Git guarda todo o histórico de alterações.
 - **A Linha do Tempo** é construída através de "pontos de salvamento" chamados **Commits**. Cada commit funciona como uma foto do estado do seu código naquele exato momento. Se você cometer um erro no futuro, pode facilmente usar esses pontos de salvamento para "viajar no tempo" e restaurar o código para uma versão anterior que estava funcionando.
-

2. Passo a Passo Prático

Passo 1: Criando a Pasta do Projeto e Abrindo o Terminal

1. Escolha um local em seu computador e crie uma nova pasta chamada **aula-1**.
2. Entre na pasta criada, clique com o **botão direito** em um espaço vazio e selecione a opção **Abrir o Terminal**. Isso abrirá o terminal do Windows já apontado diretamente para dentro da pasta do seu projeto.

Passo 2: Inicializando o Repositório (**git init**)

Para dizer ao Git que ele deve começar a monitorar essa pasta, digite o seguinte comando no terminal:

git init

- **O que acontece por trás dos panos:** O Git cria uma pasta oculta chamada **.git** dentro do seu projeto. Essa pasta armazena todas as configurações e históricos de versionamento. Para visualizá-la, você precisa ativar a opção de "itens ocultos" no gerenciador de arquivos do seu sistema.

Aviso: Nunca modifique ou apague os arquivos desta pasta manualmente!

Passo 3: Criando o Primeiro Arquivo de Código

Para testarmos o versionamento na prática, vamos criar um arquivo de simulação:

1. Dentro da pasta do projeto, crie um novo arquivo de texto chamado **meu_codigo.txt**.
2. Abra o arquivo e escreva algo simples, como: *"Olá, meu nome é [Seu Nome]"* e salve o arquivo.

Passo 4: Verificando o Estado do Projeto (**git status**)

Agora, vamos perguntar ao Git o que ele acha do nosso projeto. No terminal, execute:

git status

- **Análise do feedback do Git:** O terminal retornará um relatório informando que não existem commits feitos ainda e mostrará o arquivo `meu_codigo.txt` em **vermelho** sob a seção de "untracked files" (arquivos não monitorados). Isso significa que o arquivo existe na pasta, mas o Git ainda não o adicionou ao controle de versão.

Passo 5: Preparando o Arquivo para Salvar (`git add`)

Antes de salvar uma versão definitiva, precisamos colocar o arquivo na área de preparação (também chamada de *staging area*).

- Para preparar **apenas um arquivo específico**, digite:

```
git add "meu_codigo.txt"
```

- Se você tiver **múltiplos arquivos** (como imagens, códigos e planilhas) e quiser preparar todos de uma vez de forma rápida e eficiente, utilize o comando:

```
git add .
```

- **Verificando novamente:** Ao rodar `git status` após o comando `git add`, você verá que o arquivo agora aparece em **verde**, indicando que ele foi preparado com sucesso e está pronto para ser salvo na linha do tempo.

Passo 6: Configurando sua Identidade (Apenas na primeira vez)

Se este for o seu primeiro commit no computador, o Git precisará saber quem é o autor da alteração antes de salvar. Configure seu e-mail e nome globalmente:

```
git config --global user.email "seu_email@exemplo.com"
```

```
git config --global user.name "Seu Nome Real"
```

(Nota: Use o mesmo e-mail que você utilizou para criar sua conta no GitHub para que seus commits fiquem vinculados ao seu perfil de forma profissional).

Passo 7: Criando o Primeiro Ponto de Salvamento (`git commit`)

Agora que os arquivos estão preparados e sua identidade está configurada, vamos gravar permanentemente a primeira versão do código na linha do tempo:

```
git commit -m "Commit inicial"
```

- **O que significa o -m?** Significa "mensagem". Toda vez que criamos uma nova versão, devemos escrever uma mensagem curta entre aspas que explique de forma clara o que foi feito ou adicionado nessa versão do código.

Se você rodar o comando `git status` agora, o Git informará que não há nada pendente para salvar, pois seu histórico de trabalho local está totalmente atualizado!

Passo 8: Conectando à Nuvem e Enviando o Código (`git remote` e `git push`)

Agora que você já tem o seu "ponto de salvamento" criado localmente no seu computador, o último passo é enviar essa linha do tempo para a sua conta no GitHub, garantindo um backup seguro e um link para o seu portfólio.

1. **Crie o repositório no GitHub:** Acesse sua conta no GitHub, clique no botão "New" para criar um novo repositório. Dê o nome de **aula-1** e clique em *Create repository*.
Importante: Deixe o repositório totalmente vazio (não adicione README, licença ou .gitignore agora).
2. **Conecte o repositório local com o remoto (git remote):** O GitHub vai te mostrar uma tela com alguns comandos. Volte ao seu terminal e diga ao Git qual é o endereço da sua nuvem colando o comando similar a este:

git remote add origin https://github.com/[SeuUsuario]/aula-1.git

- **O que isso significa?** Você está apenas apresentando o GitHub ao seu computador. "Origin" é o apelido padrão que damos a esse link da internet.

3. **Envie o seu código — O grande momento! (git push):** Agora, vamos "empurrar" seus arquivos e o seu histórico para o servidor. No terminal, execute:

git push -u origin master

(Nota: Se o seu terminal indicar que a branch principal se chama "main" ao invés de "master", utilize ***git push -u origin main***).

- **O que acontece por trás dos panos:** O comando **push** faz o upload de tudo o que foi salvo no Passo 7. A parte **-u** (upstream) cria uma ligação definitiva entre o seu computador e o GitHub. Isso significa que, nas próximas vezes que você quiser enviar novos commits desse mesmo projeto, bastará digitar apenas **git push**, sem precisar repetir o link!

Vá até a página do seu repositório no GitHub, atualize a página (F5) e comemore: seu código já está na nuvem!

3. Desafio Prático da Aula: A Corrente do Git (Dinâmica em Grupo)

Objetivo do Desafio: Criar uma lista colaborativa em formato **.txt** onde cada aluno adiciona seu nome e suas credenciais. O objetivo é que cada participante faça o clone do projeto, configure sua própria identidade local no terminal, realize a alteração, faça o commit e envie de volta ao servidor para que o próximo colega continue a corrente.

Fase 1: A Preparação (Realizada pelo Professor)

Antes de iniciar a corrente, um repositório centralizado precisa ser criado para servir como o servidor remoto do grupo.

1. **Criar o Repositório no GitHub:** Acesse o GitHub, crie um novo repositório público chamado **lista-turma**.
Adicionar o Arquivo Inicial: No seu computador, crie uma pasta, inicialize o repositório com **git init** e crie um arquivo chamado **alunos.txt** com o seguinte cabeçalho:
=== LISTA DE ALUNOS - CURSO GIT E GITHUB ===
2. **Enviar ao GitHub:** Adicione o arquivo (**git add alunos.txt**), faça o commit inicial (**git commit -m "Commit inicial: Cria arquivo alunos.txt"**) e suba para o GitHub (**git push**).
3. **Liberar Acesso aos Alunos:** No GitHub, vá em *Settings > Collaborators* e adicione os usuários do GitHub de todos os alunos da equipe para que eles tenham permissão de realizar o **push** diretamente no repositório.

Fase 2: O Fluxo de Trabalho de Cada Aluno (A Corrente)

Os alunos devem realizar as etapas de forma **sequencial** (um por vez). Quando o Aluno 1 terminar, ele avisa o Aluno 2, que repete o processo, e assim por diante.

Cada aluno da vez deve abrir o terminal e executar rigorosamente os seguintes passos:

Passo 1: Clonar o Repositório

Vá até a pasta onde deseja trabalhar e clone o projeto do GitHub para a sua máquina local:

```
git clone https://github.com/[NOME_DO_LIDER]/lista-turma.git
```

*(Nota: Esse comando criará uma pasta local contendo o arquivo **alunos.txt** criado pelo professor).*

Passo 2: Configurar a Identidade Local

Para garantir que o histórico de commits registre exatamente quem realizou a alteração, o aluno precisa configurar seu próprio nome e e-mail no Git local antes de salvar qualquer coisa:

```
git config user.email "seu_email_do_github@exemplo.com"
```

```
git config user.name "Seu Nome Completo"
```

(Atenção: Use o mesmo e-mail e nome cadastrados na sua conta do GitHub).

Passo 3: Atualizar o Arquivo Local (Muito Importante!)

Antes de fazer qualquer alteração, o aluno deve garantir que tem a versão mais recente do arquivo na sua máquina. Embora a corrente seja sequencial, o comando abaixo é uma boa prática indispensável para evitar conflitos no futuro:

```
git pull origin master
```

*(Nota: Se o repositório principal usar a nomenclatura mais recente do GitHub, substitua **master** por **main**).*

Passo 4: Editar o Arquivo

Abra o arquivo **alunos.txt** em um editor de texto e adicione os seus dados no final da lista, seguindo o padrão estabelecido:

- Nome: [Seu Nome Completo] | Usuário Git local: [Nome configurado no Passo 2] | E-mail: [Seu E-mail] |
GitHub: @[SeuUsuarioGitHub]

Salve e feche o arquivo.

Passo 5: Verificar o Status e Preparar o Envio

Abra o terminal dentro da pasta do projeto e verifique se o Git detectou a sua alteração:

```
git status
```

O arquivo **alunos.txt** aparecerá listado como modificado em vermelho. Adicione-o à área de preparação (*staging area*):

```
git add alunos.txt
```

Passo 6: Fazer o Commit

Grave sua alteração de forma definitiva na linha do tempo do repositório local:

```
git commit -m "[Seu usuário github]"
```

Passo 7: Enviar para o Servidor Remoto (GitHub)

Suba sua alteração para o GitHub para que os próximos colegas possam ter acesso ao arquivo atualizado:

```
git push origin master
```

(Ou *git push origin main*, se for a branch principal).

Passo 8: Passar o Bastão

O aluno deve acessar o repositório no GitHub para confirmar se seu commit aparece na página e se os seus dados estão no arquivo [alunos.txt](#). Com tudo confirmado, ele deve avisar o próximo aluno da lista que a vez dele começou!